

源文件: python/README.md

Python 数据、特征与 BP 训练教程

`python/` 保存从六轴 IMU 数据到 ESP32 模型工件的完整算法源码：文件级划分、窗口清洗、297 项特征、双轻量 BP、C 导出和文档图生成。

从哪里开始

想做什么	入口
看懂数据到模型的完整流程	<code>train_export.py</code>
复现四张算法图	<code>visualize_action_features.py</code>
准备清单约束的数据集	<code>prepare_finals_dataset.py</code>
导出双模型到 ESP32	<code>dual_m0_export.py</code>

算法公式、每项特征的物理意义和资源影响见[算法原理](#)、[训练与实时计数](#)。

从数据到 C 模型的执行顺序

步骤	为什么要做	做完有什么效果	关键检查
1. 建立项目虚拟环境	隔离 NumPy、PyTorch 和绘图库版本，避免系统 Python 的包漂移改变训练或图表	其他读者能在仓库根使用同一解释器和依赖集合复现入口	Python 版本、依赖安装退出码
2. 扫描数据并验证合同	目录名、列数、有限值、通道顺序和单位是所有后续计算的前提	错数据在训练前失败，不会悄悄变成错误标签或异常特征	每类文件数、拒绝文件原因
3. 先按文件划分	相邻重叠窗口高度相似，先切窗再随机分组会泄漏	验证和测试更接近新会话表现	同一文件 ID 只属于一个角色
4. 只用训练角色拟合前处理	活动阈值、标准化参数若查看验证/测试，会把答案带回训练	验证和测试保持独立，指标可解释	阈值、均值和标准差的来源角色
5. 先生成特征图	曲线和分布能在训练前暴露单位错误、饱和、弱类重叠和异常文件	开发者能直接检查模型输入中的物理差异	图表清单中的数据指纹和文件等权规则
6. 训练并按验证集选模	训练损失只说明拟合，宏 F1 和最弱类更能暴露类别不平衡	保存的是泛化折中更好的检查点，而非最后一个 epoch	准确率、宏 F1、逐类召回、早停原因
7. 冻结后做独立测试	测试集不能反复参与参数选择	最终指标代表一次确认，不成为新的调参反馈	测试只在所有选择冻结后运行
8. 导出参数、清单和 C 头	ESP32 需要权重之外的类别、特征、标准化和融合合同	端侧得到一套自描述、可校验的模型工件	维度、顺序、SHA-256、类别 CRC

步骤	为什么要做	做完有什么效果	关键检查
9. Python/C 逐值对照	同一公式在浮点库、循环顺序或边界实现上可能有差异	训练结论真正落到 ESP32 计算路径	特征误差、logits 误差和 Top-1 一致

这个顺序让错误尽早暴露并停在明确层级。图表已经显示单位异常时停止训练；Python/C 特征不一致时先修部署实现；测试集变差时保留结果，不能回头反复挑参数直到测试变好。

10 分钟数据路径

从仓库根目录执行：

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r python\requirements.txt
```

POWERSHELL

把数据放到：

```
IMU_Dataset/
├─ imu_dataset_for_final/
│  ├─ good_morning/
│  ├─ jumping_jack/
│  ├─ ...
└─ wave/
```

TEXT

先复现文档图，不训练模型：

```
.\.venv\Scripts\python.exe -m python.visualize_action_features `
--dataset-dir IMU_Dataset\imu_dataset_for_final `
--output-dir docs\assets\algorithm
```

POWERSHELL

正式输出固定为：

```
docs/assets/algorithm/
├─ 01_六类派生信号曲线.png
├─ 02_十一类功率谱对比.png
├─ 03_关键特征分布.png
├─ 04_特征中位数热力图.png
└─ figure_manifest.json
```

TEXT

图表清单不写本机绝对路径和生成时间；相同数据、代码和参数应产生相同统计事实。清单记录数据指纹、文件级划分、窗口参数、阈值、文件级聚合方法和限制。

数据合同

每个动作 TXT 为 N×8：

```
gx, gy, gz, ax, ay, az, timestamp_low, timestamp_high
```

TEXT

- 通道顺序固定为 `gx、gy、gz、ax、ay、az`。
- 陀螺仪原始值除以 `16.4`，换算为 `deg/s`。
- 加速度原始值除以 `4096`，换算为 `g`。
- 基础数据采样率为 25 Hz，最后两列当前为 0。
- 类别由第一层目录名确定；新增未知目录会改变类别顺序，必须同步模型和 ESP32 清单。

基础数据当前包含 11 类、189 个动作文件。部分高动态文件包含“动作—休息—动作”，因此脚本需要活动段裁剪和窗口筛选，不能把整文件都当作连续标准动作。

数据没有完整受试者 ID 和佩戴侧元数据。文件级划分可以阻止同一文件的重叠窗口跨集合，但不能证明严格跨人员或左右手泛化。

数据流

```
scan_dataset
-> split_records_by_file
-> load_imu_file
-> 单位换算
-> 训练文件估计静止/活动阈值
-> 动态记录首尾活动段裁剪
-> 62 点窗口、0.5 秒步长
-> 标签相关活动筛选
-> 单轴孤立尖峰清洗
-> build_feature_series
-> extract_features (297 维)
-> 训练集标准化
-> 六分支 BP
-> 验证选择与测试确认
-> 双模型包和 C 头文件
```

TEXT

关键原则：

1. 先按文件划分，再生成窗口；不能先切窗再随机划分。
2. 静止阈值只从训练划分估计；验证和测试不得反向改变阈值。
3. 标准化均值和标准差只从训练特征拟合。
4. 数据增强只作用于训练窗口。
5. 验证用于检查配置；测试和外部留出只用于当前配置的独立确认。

为什么图表按文件等权

基础记录长度差异明显。同一长文件会产生更多重叠窗口；如果直接把所有窗口当独立样本，长记录会支配箱线图、热力图和类中心。

文档图采用两级聚合：

1. 在每个验证文件内聚合该文件的有效窗口；
2. 再对同类文件计算中位数和 25% ~ 75% 分位区间。

这样每个采集文件权重相同，同时保留文件之间的节奏、幅度和动作差异。时域图使用重力相对垂直加速度与角速度模长，不直接跨佩戴姿态平均 `gx/gy/gz`。功率谱先去均值、加 Hann 窗，再转成相对功率，降低幅值和相位的直接影响。

四张图：

- 六种动作时域特征
- 十一类相对功率谱
- 关键特征分布
- 动作—特征热力图

完整方法和数据指纹见[图表可复现清单](#)。

计数曲线与事件回放

计数教程组合三类互补材料：

1. 项目数据集六类时域曲线来自 `imu_dataset_for_final` 的固定文件级验证划分，用于观察开合跳、深蹲、跳跃深蹲、跳跃弓步、挥手和行走的周期形态及文件间差异。
2. 实时计数算法示意曲线由固件生产常量生成，用于解释自适应端点、完整闭合、步峰重武装和休息冻结；它明确标记为非实测。
3. 真板深蹲计数事件回放只使用现场 CSV 中的物理量、设备状态和权威 `MetricEvent`，用于核对每次加一的实际时刻。

项目数据集没有逐次计数标签，因此生成器不会在数据集曲线上伪造事件。要回放自己的 44 列上位机日志：

```
.\.venv\Scripts\python.exe python\visualize_counting_process.py `
--field-log logs\IMU_全部缓存.csv `
--session-id 81 `
--output-dir docs\assets\algorithm `
--manifest docs\assets\algorithm\counting_curve_manifest.json`
```

POWERSHELL

生成器会检查会话、设备单调时间、六轴物理量和累计事件连续性。公开清单只记录源文件名、SHA-256、会话摘要、生产常量和输出哈希；原始现场 CSV、蓝牙地址和设备标识不进入图片或仓库。

297 项特征

当前输入按固定顺序分成六组：

分组	维数	主要信息
全局序列统计	112	六轴、模长、差分和重力相对序列的均值、标准差、范围、RMS、跳变和过零
原始四阶段	48	动作窗口四个时间阶段的均值、标准差和最大绝对值
时序与周期	24	活动比例、峰数、主频、谱熵、自相关峰和周期
归一化四阶段	48	去除平移和尺度后的阶段形状
冲击分布	32	分位数、偏度、峰度和最大跳变
弱类机制	33	频带、峰形、互相关、腾空/落地、各向异性和手腕换向

六轴派生序列包括：

- `gyro_mag`、`acc_mag`：方向不敏感的模长；
- `gyro_delta_mag`、`acc_delta_mag`：相邻点变化；
- `acc_vertical`、`gyro_vertical`：沿窗口重力方向的分量；
- `acc_horizontal_mag`、`gyro_horizontal_mag`：垂直于重力方向的模长。

`build_feature_names()` 和 `extract_features()` 必须始终同长度、同顺序。ESP32 生成头的顺序也必须完全一致。

训练

完整训练：

```
.\.venv\Scripts\python.exe -u python\train_export.py `
--dataset-dir IMU_Dataset\imu_dataset_for_final
```

POWERSHELL

逐 epoch 日志包含：

- 总损失和交叉熵；
- 可选的跨文件监督对比、定向 margin 和辅助任务损失；
- 验证准确率和宏平均 F1；
- 11 类逐类召回率；
- 当前最弱类别和召回率；
- 最佳 epoch、早停计数和恢复状态。

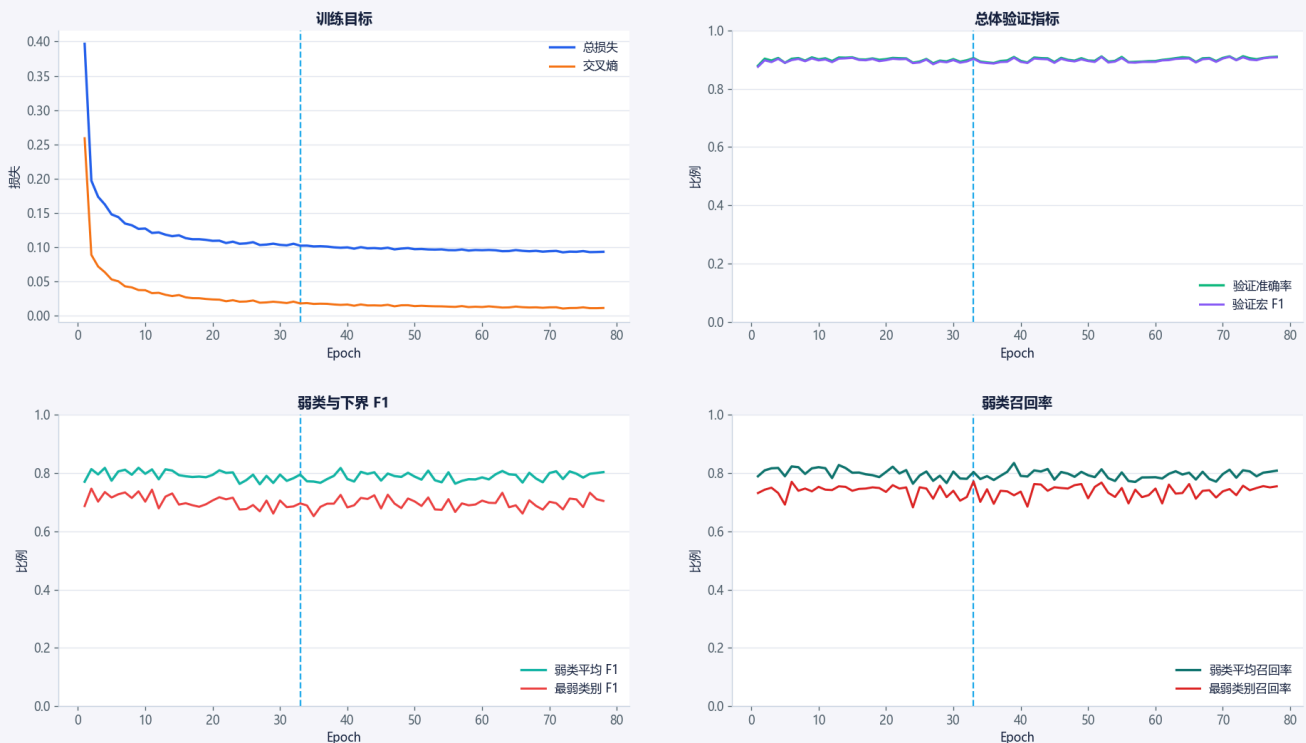
```

IMU_BP_Visible_Training
val_worst_f1=0.7468 val_weak_recall=0.8095 val_min_recall=0.7432 best_epoch=2 patience_left=45
window=2.5s epoch=003 loss=0.1737 ce=0.0721 supcon=1.8860 margin=0.0290 val_acc=0.8971 val_f1=0.8918 val_weak_f1=0.7957
val_worst_f1=0.7022 val_weak_recall=0.8166 val_min_recall=0.7500 best_epoch=3 patience_left=45
window=2.5s epoch=004 loss=0.1625 ce=0.0637 supcon=1.8368 margin=0.0278 val_acc=0.9059 val_f1=0.9029 val_weak_f1=0.8180
val_worst_f1=0.7350 val_weak_recall=0.8178 val_min_recall=0.7317 best_epoch=3 patience_left=44
window=2.5s epoch=005 loss=0.1480 ce=0.0532 supcon=1.7849 margin=0.0224 val_acc=0.8894 val_f1=0.8892 val_weak_f1=0.7741
val_worst_f1=0.7163 val_weak_recall=0.7894 val_min_recall=0.6919 best_epoch=3 patience_left=43
window=2.5s epoch=006 loss=0.1441 ce=0.0505 supcon=1.7598 margin=0.0224 val_acc=0.9031 val_f1=0.8987 val_weak_f1=0.8059
val_worst_f1=0.7274 val_weak_recall=0.8230 val_min_recall=0.7703 best_epoch=6 patience_left=45
window=2.5s epoch=007 loss=0.1347 ce=0.0431 supcon=1.7369 margin=0.0192 val_acc=0.9056 val_f1=0.9027 val_weak_f1=0.8115
val_worst_f1=0.7337 val_weak_recall=0.8203 val_min_recall=0.7394 best_epoch=6 patience_left=44
window=2.5s epoch=008 loss=0.1322 ce=0.0416 supcon=1.7239 margin=0.0179 val_acc=0.8968 val_f1=0.8947 val_weak_f1=0.7946
val_worst_f1=0.7147 val_weak_recall=0.7973 val_min_recall=0.7471 best_epoch=6 patience_left=43
window=2.5s epoch=009 loss=0.1269 ce=0.0377 supcon=1.7082 margin=0.0151 val_acc=0.9081 val_f1=0.9044 val_weak_f1=0.8183
val_worst_f1=0.7375 val_weak_recall=0.8165 val_min_recall=0.7375 best_epoch=6 patience_left=42
window=2.5s epoch=010 loss=0.1274 ce=0.0375 supcon=1.7041 margin=0.0187 val_acc=0.9011 val_f1=0.8977 val_weak_f1=0.7978
val_worst_f1=0.7021 val_weak_recall=0.8204 val_min_recall=0.7529 best_epoch=6 patience_left=41
window=2.5s epoch=011 loss=0.1210 ce=0.0332 supcon=1.6925 margin=0.0129 val_acc=0.9047 val_f1=0.9008 val_weak_f1=0.8127
val_worst_f1=0.7432 val_weak_recall=0.8169 val_min_recall=0.7432 best_epoch=6 patience_left=40
window=2.5s epoch=012 loss=0.1218 ce=0.0337 supcon=1.6901 margin=0.0143 val_acc=0.8955 val_f1=0.8917 val_weak_f1=0.7789
val_worst_f1=0.6789 val_weak_recall=0.7830 val_min_recall=0.7419 best_epoch=6 patience_left=39
window=2.5s epoch=013 loss=0.1184 ce=0.0308 supcon=1.6854 margin=0.0133 val_acc=0.9076 val_f1=0.9035 val_weak_f1=0.8130
val_worst_f1=0.7194 val_weak_recall=0.8277 val_min_recall=0.7548 best_epoch=6 patience_left=38
window=2.5s epoch=014 loss=0.1163 ce=0.0290 supcon=1.6804 margin=0.0131 val_acc=0.9070 val_f1=0.9046 val_weak_f1=0.8088
val_worst_f1=0.7303 val_weak_recall=0.8176 val_min_recall=0.7529 best_epoch=6 patience_left=37
window=2.5s epoch=015 loss=0.1175 ce=0.0304 supcon=1.6795 margin=0.0127 val_acc=0.9082 val_f1=0.9065 val_weak_f1=0.7931
val_worst_f1=0.6920 val_weak_recall=0.8014 val_min_recall=0.7394 best_epoch=6 patience_left=36
window=2.5s epoch=016 loss=0.1134 ce=0.0271 supcon=1.6699 margin=0.0114 val_acc=0.9011 val_f1=0.8992 val_weak_f1=0.7893
val_worst_f1=0.6968 val_weak_recall=0.8020 val_min_recall=0.7459 best_epoch=6 patience_left=35

```

模型训练过程（历史候选：184 维，2.5 秒窗口，78 个 epoch）

蓝色虚线：验证规则选出的最佳 epoch 33；测试集不参与建模



截图与曲线来自同一份真实历史候选日志：184 维特征、2.5 秒窗口、78 个 epoch，验证规则选择第 33 个 epoch。曲线没有平滑或补齐指标，四个面板分别显示训练目标、总体验证指标、弱类 F1 和弱类召回率。它说明怎样观察训练和早停，不代表当前部署模型；该候选记录了 `target_reached=false` 与 `header_export_skipped=true`，没有进入固件。

当前部署仍以 `dual_m0_manifest.json` 的 297 维双 M0 为准。训练曲线的输入日志 SHA-256 和候选摘要位于 `training_curve_manifest.json`。

训练完成后，可对任意同格式日志生成自己的曲线：

POWERSHELL

```
.\.\venv\Scripts\python.exe python\visualize_training_history.py `
--log outputs\training_console.log `
--output outputs\training_curve.png `
--manifest outputs\training_curve_manifest.json
```

生成器严格读取日志中的总损失、交叉熵、准确率、宏 F1、弱类指标、最弱类别指标、最佳 epoch 和早停状态。输入日志字段缺失、epoch 重复或编码损坏时直接失败，不生成看似正常的图片。

只运行验证模式、不触碰测试：

POWERSHELL

```
.\.\venv\Scripts\python.exe -u python\train_export.py `
--dataset-dir IMU_Dataset\imu_dataset_for_final `
--validation-only `
--window-seconds 2.5
```

`--validation-only` 不构建测试窗口、不输出测试指标，也不导出 ESP32 正式头文件。验证模式通过预声明门后，才执行独立测试确认。

面向动作差异的稳健增强

`--augmentation-profile baseline` 提供基础增强。 `--augmentation-profile robust` 覆盖更宽的右腕执行差异：

- 六轴同步刚体旋转，覆盖有限佩戴角变化；
- 动态幅度缩放，保持加速度平均重力；
- 固定 62 点输出下的持续时间缩放和局部非匀速扭曲；
- 轻微传感器噪声。

增强用于覆盖“同一个动作做得略快、略慢、幅度略大或略小”，不是把非目标动作变成目标动作。它不包含左右手镜像，也不能替代独立用户数据。

当前轻量模型合同

每个 M0 把 297 维输入分成 `(112, 48, 24, 48, 32, 33)` 六组，独立编码后融合。部署使用两个同构 M0：

- 基础模型读取全部标准化特征；
- 掩码模型把标准化索引 `184:232` 的相位敏感组固定为零；
- 两个 logits 使用固定权重融合；
- ESP32 训练引擎再完成主动作确认和实时计数许可。

训练期辅助头只帮助学习，不进入 ESP32 主推理路径。教程只说明当前双 M0 配置；网络、特征和实时决策的完整定义见[算法文档](#)。

导出到 ESP32

正式产物位于 `esp32/include/`：

```
dual_m0_bundle.npz  
dual_m0_manifest.json  
esp32_bp_features.h  
esp32_dual_m0_model.h
```

TEXT

- `dual_m0_bundle.npz`：两套权重、偏置、标准化参数和掩码；
- `dual_m0_manifest.json`：类别、维度、融合权重、文件哈希和生成合同；
- `esp32_bp_features.h`：清洗、297 项特征和基础时间逻辑；
- `esp32_dual_m0_model.h`：两个六分支前向、掩码和融合。

查看导出参数：

```
.\.venv\Scripts\python.exe python\dual_m0_export.py --help
```

POWERSHELL

发布前必须核对 `dual_m0_manifest.json` 中的 297 项特征、两个模型、固定融合权重和生成文件 SHA-256；部署必须以清单与同目录工件为准。

数据准备与发布边界

- `prepare_finals_dataset.py`：按 `finals_jumping_squat_manifest.json` 校验补充会话哈希、行数和角色。

外部真板 CSV 只用于独立确认，不参与增强范围、模型、融合权重或阈值选择。

模型适用边界

- 文件级划分阻止同一文件泄漏，但没有受试者 ID，不能声称严格跨人泛化。
- 没有可靠佩戴侧标签；当前产品固定右手腕，不承诺左腕。
- 文档图只使用固定验证划分，不读取测试集，不用于调参。
- 数据增强覆盖有限执行差异，不代表所有体型、力量、疲劳和非标准姿态。
- C/Python 一致性证明实现对齐，不证明传感器、BLE、触摸和功耗已经通过。
- 最终发布仍需目标人群、独立会话、不同速度和幅度的真板盲测。

完整发布门、真板验收方法和当前验证边界见[测试验收与故障排查](#)。